Machine Learning in Computational Biology

Spring 2026 • Assignment #1

Epigenetic Age Prediction from DNA Methylation

Assignment Overview

This assignment develops a complete machine learning pipeline for **epigenetic age prediction** from DNA methylation data. DNA methylation — the addition of methyl groups to cytosine bases at CpG dinucleotides — changes systematically with age across the human genome. This signal can be measured with microarray technology and used to train regression models that predict chronological age from methylation profiles. Such models are called **epigenetic clocks**, and have broad applications in biology and medicine: measuring biological ageing rate, identifying accelerated ageing in disease, and understanding the epigenetic basis of lifespan.

The assignment is structured into **four graded tasks** plus **two optional bonus tasks**. All tasks build incrementally on each other: the dataset and preprocessing infrastructure established in Task 1 feeds into the baseline models in Task 2, the feature selection in Task 3, and the final tuned models in Task 4.

The Dataset

Source

You will work with **GSE40279** (Hannum et al., 2013), a publicly available dataset measuring whole-blood DNA methylation using the Illumina 450K microarray. The original dataset comprises 656 samples from individuals aged 19–101 years. The 450K array measures beta values — the proportion of methylated reads at each CpG site, ranging from 0 (fully unmethylated) to 1 (fully methylated) — at approximately 480,000 CpG positions.

For this assignment, the dataset has been pre-processed into the following files, available on GitHub:

File	Samples	Purpose
`development_data.csv`	456	All model development: preprocessing, feature selection, training, cross-validation tuning.
`evaluation_data.csv`	100	Final evaluation only. Do not touch during training, feature selection, or tuning.

Each row is a sample (labelled by GEO sample ID). Each column is either a **CpG feature** (named `cg`XXXXXXXX), an **age** column (integer years, the regression target), a **sex** column (F/M), or an **ethnicity** column. The 1000 CpG features included in the datasets were selected from the full 480K array by absolute Spearman correlation with age across all samples.

Preprocessing requirements

To simulate real-world data quality, **some of the CpG values have been randomly set to NaN**. You must address this before training models. Please note that all parameter estimation steps needed for dataset preprocessing **must use exclusively the training split** of the development dataset and then applied to the validation split and the evaluation dataset. Fitting on the full dataset constitutes **data leakage** (Read IBM's article on train-test contamination) that will lead to optimistic results (...and a grade reduction).

Task 0: Environment Setup [Non-graded]

Complete the following environment setup before beginning the graded tasks.

1. Select and install an IDE (Visual Studio Code recommended).
2. Windows users: set up a Linux environment via WSL2.
3. **Install Python** via Anaconda or Miniconda in your terminal environment.
4. Create a **GitHub repository** for this assignment and initialise it with a README.
5. Structure your repository as follows:
   - `src/` — all Python source files (`functions.py`, etc.)
   - `notebooks/` — Jupyter notebooks
   - `data/` — dataset files (**do not commit data to GitHub**; add to .gitignore)
   - `models/` — saved model files
   - `figures/` — generated plots

If you are not familiar with GitHub, you should start familiarizing yourself with the platform and Git, as it is widely used among Data Scientists and engineers. A good starting point is this tutorial for beginners, which will guide you through the basics and also point you to more advanced resources.

Jupyter Notebooks are also a handy tool for Data Scientists as it allows you to interactively explore your data and develop your models. It combines code, visualizations, text and other rich media in a single document, creating a cohesive workflow. This guide will give you a step-by-step walkthrough from installing to sharing your work.

Task 1: Data Loading and Preprocessing [15 points]

Create a notebook `notebooks/data_exploration.ipynb` and a codebase file `src/functions.py`. In `functions.py` implement all reusable pipeline functions. In the notebook, demonstrate your preprocessing pipeline and exploratory analysis.

It is a good practice to save the functions you build on your own and might reuse in the future in codebase files so you do not have to reinvent the wheel each time you stumble upon a problem. In the notebook, demonstrate your preprocessing pipeline and exploratory analysis.

1.1 Data splitting [5 pts]

Load `development_data.csv` and split it into an internal **80% training / 20% validation** split (stratified by age, seed=42). Load `evaluation_data.csv` separately and **treat it as locked** — it must not influence any design decision.

1.2 Preprocessing [5 pts]

Implement a preprocessing pipeline applied consistently across all splits. All steps must be **fit exclusively on the training split** and then applied to the validation (and later to the evaluation data) without re-fitting to avoid data leakage. Similarly, when doing cross validation in later tasks, remember that we always fit on the training split and transform the validation split.

- **Missing values:** Inspect the data for missing values. What strategy is appropriate for data imputation given that values are MCAR (Missing Completely at Random)? Consider how your choice might affect downstream models.
- **Feature scaling:** apply StandardScaler to CpG features inside a sklearn Pipeline
- **Categorical features:** Some metadata columns are categorical (sex, ethnicity). Consider how to represent them numerically — and whether including any of these variables provides a predicting signal for age.

For Task 2, you will need to evaluate how much predictive signal comes from CpG features versus metadata. Think about what **combinations of feature groups** would make meaningful baselines — for instance, what does a model using only metadata tell you? What does adding CpG features change? Construct the feature matrices you need to answer these questions.

1.3 Exploratory analysis [5 pts]

Provide in your notebook:

- A summary of dataset dimensions, age distribution (mean, std, range), and class balance for sex.
- A visualisation of the age distribution for the development set.
- A brief justification of any additional cleaning steps you performed.

Deliverable (Task 1): In your report, include a table of dataset statistics (n per split, age mean ± std, sex balance) and one age distribution figure. Justify your missing value strategy, your handling of categorical features, and the feature matrices you chose to construct.

Task 2: Baseline Models [20 points]

Before any feature selection or tuning, establish **performance baselines** that serve as the minimum benchmark for subsequent tasks.

2.1 OLS LinearRegression across feature sets [8 pts]

Train plain **OLS LinearRegression** (ordinary least squares, no regularisation). You can learn more about OLS on datacamp, and go through the scikit-learn documentation to implement it on the feature matrices you constructed in Task 1.2.

OLS Linear Regression provides a simple, interpretable, and unregularized baseline that allows us to evaluate whether and how much more complex and regularized models improve predictive performance.

Train each model **once using the training set (as split in Task 1.1).**

Do not use cross-validation or hyperparameter tuning for the baselines.

Evaluate each trained model on the validation set using **bootstrap resampling** of the validation set**(1000 resamples, seed=42)**. Bootstrap must be applied only to the validation predictions.

Report **RMSE, MAE, R^2, and Pearson r with 95% confidence intervals (CI)**.

2.2 Three regression models at default hyperparameters [12 pts]

Train the following three algorithms at **scikit-learn default hyperparameters** on the CpG-only feature set (1000 features):

- **ElasticNet** — L1+L2 regularised linear regression
- **SVR** — Support Vector Regression with RBF kernel
- **BayesianRidge** — Bayesian linear regression with automatic relevance determination

Evaluate each one on the validation set with the same bootstrap protocol (1000 resamples, 95% CI). Every model must beat the OLS baseline to justify its inclusion in subsequent tasks.

> **Deliverable (Task 2):** Report a table with RMSE mean, 95% CI, MAE, R^2, and Pearson r for all baselines. Include bootstrap boxplots for RMSE and R^2 comparing all models. Comment on whether each model beats the OLS floor and why.

Task 3: Feature Selection [25 points]

Reducing the 1000 CpG features to a smaller, more informative subset can improve generalization by removing noisy or redundant probes. Implement and compare **two feature selection methods**, then proceed with the better-performing set. For more on feature selection visit this page.)

3.1 Stability Selection [10 pts]

Stability selection estimates the robustness of feature rankings across data perturbations:

1. Draw 50 subsamples (each one has size 80% of training data, without replacement).
2. For each subsample, rank all CpG features by |Spearman correlation| with age and keep the top 200.
3. A feature is considered stable if it appears in more than half of the 50 resamples.
4. Report the number of stable features selected and their selection-frequency distribution.

Rationale: Stability selection is robust to training-set perturbations. Its weakness is that it ignores inter-feature redundancy — correlated CpGs that measure the same biological signal may all be selected together.

3.2 Minimum Redundancy Maximum Relevance (mRMR) [10 pts]

The mRMR method (mechanism and Python examples can be found here) addresses the redundancy limitation by jointly optimising two objectives:

- **Maximum Relevance:** each selected feature should have high correlation with the target variable (age)
- **Minimum Redundancy:** selected features should be minimally correlated with each other

Use the `mrmr-selection` package (`pip install mrmr-selection`) or other mRMR algorithm implementations to select K features from the training set. **Choose an appropriate value of K** — consider the trade-off between retaining enough signal and avoiding redundancy. How could you use the validation set to inform this choice? Justify your K in your report. Which value of K would we prefer in case they perform similarly? Report the top 10 selected features and their importance scores. Visualise the overlap between the stability-selected and mRMR-selected feature sets.

3.3 Method comparison and selection [5 pts]

Evaluate both feature sets using a **proxy model** on the **validation set** (not the evaluation set — that remains locked). Select the feature set with lower validation RMSE to carry forward into Task 4. Report the comparison.

Deliverable (Task 3): Table comparing Stability vs mRMR (number of features selected, chosen K with justification, validation RMSE, validation R^2). Venn diagram or bar chart of feature overlap between the two methods. State which method and K were selected and why. Report the top 10 mRMR features by importance score.

Task 4: Hyperparameter Tuning and Final Evaluation [40 pts]

4.1 Hyperparameter tuning [15 pts]

Using the selected feature set from Task 3, tune the hyperparameters of all three algorithms (ElasticNet, SVR, BayesianRidge) using **RandomizedSearchCV** with **5-fold cross-validation** on the **development set only**. Use seed=42 throughout. The objective to minimize is mean cross-validation RMSE.

Search spaces to use (minimum — you may expand but not reduce them):

Model	Key hyperparameters to search
`ElasticNet`	`alpha` (log-uniform, 0.001–10) · `l1_ratio` (uniform, 0.1–1.0)
`SVR`	`C` (log-uniform, 0.1–500) · `epsilon` {0.01, 0.1, 0.5, 1.0} · `kernel` {rbf, linear}
`BayesianRidge`	`alpha_1`, `alpha_2`, `lambda_1`, `lambda_2` (log-uniform, 1e-7–1e-3)

Use at least 40 random iterations (trials) per model. Refit the best hyperparameter combination on the **full development set** (train + validation, 456 samples) before final evaluation.

4.2 Final model evaluation [15 pts]

Evaluate all three tuned models on the **evaluation set** using **bootstrap resampling (1000 resamples, seed=42)** of the evaluation set. Report for each model:

- **RMSE** — Root Mean Squared Error (primary metric)
- **R^2** — coefficient of determination
- **MAE** — Mean Absolute Error
- **Pearson r** — linear correlation coefficient

For each metric report: mean, standard deviation, and 95% confidence interval (percentile bootstrap).

Required table format: Your report must include the following table for each task stage (baseline, FS-only, FS+tuned):

Model	Stage	RMSE (mean)	95% CI	MAE	R^2	Pearson r
ElasticNet	E.g. FS+Tuned		[lo, hi]			

Repeat this table for SVR and BayesianRidge. Include the same table for baselines (Task 2) and FS-only (Task 3) for comparison.

4.3 Model selection and final model [10 pts]

Based on your evaluation results, select and justify **one "best model"**. Justify your choice considering: (1) bootstrap RMSE and CI width, (2) model complexity and risk of overfitting given the dataset size, (3) interpretability of the learned parameters. Save the final model to `models/best_model.pkl`.

Deliverable (Task 4): All tables in the format shown above. Bootstrap boxplots for each metric across all three models. A scatter plot of Predicted vs Actual Age for each tuned model on the evaluation set. A written justification of model selection (minimum 150 words).

Bonus A: Optuna Hyperparameter Optimization [20pts]

Add an alternative hyperparameter tuning method to your `functions.py` using **Optuna** (`pip install optuna`). Optuna implements Bayesian optimisation via the **Tree-structured Parzen Estimator (TPE)**: unlike random search, TPE builds a probabilistic surrogate model of the objective function and samples more densely from regions of the search space that produced good results in previous trials.

Implementation requirements

- Implement `optuna_tune_model(model_name, pipeline, X_train, y_train, n_trials, cv=5)` that returns the best pipeline.

Evaluation

Apply Optuna tuning to all three models and compare against RandomizedSearchCV results:

- Report a head-to-head table: model name, RandomSearch RMSE, Optuna RMSE, delta, winner.
- Include a figure showing the Optuna optimisation history (trial RMSE vs trial number) for one model.
- Discuss: Do the results converge to similar values on this dataset? Under what conditions does Optuna offer a larger advantage?

Bonus B: Sex Prediction from DNA Methylation [30pts]

Biological background

DNA methylation is closely linked to sex biology. In females, one of the two X chromosomes is permanently silenced in every somatic cell via a process called **X-inactivation**. This silencing is mediated by dense methylation of CpG sites near the **XIST** gene (X Inactive Specific Transcript) and hundreds of other X-linked loci. Males, having only one X chromosome, show entirely different methylation patterns at these probes — beta values at XIST-linked CpGs are typically 0.7–0.9 in females and 0.05–0.15 in males.

This creates a strong, biologically interpretable classification signal that is **largely independent of the age signal**: the CpGs most predictive of sex are different from those most predictive of age. Predicting sex from methylation data is therefore an instructive contrast to the age regression task — the signal is binary and concentrated rather than continuous and distributed.

Task description

Convert the dataset into a **binary classification problem** by using the **sex** column as the target label (F = 0, M = 1). Implement a classification pipeline in `functions.py` and evaluate it in a new notebook section or cell.

Feature selection for sex

Use **mRMR** (as implemented in Task 3) with K features as before, this time targeting the sex label instead of age. Visualise the overlap between the age-selected and sex-selected CpG sets — you should find that they are largely non-overlapping. The 1000 CpGs pre-selected for age correlation are **not** necessarily the best features for sex prediction.

Models

Train the following two classifiers (no hyperparameter tuning required for the bonus):

- **Logistic Regression** — linear classifier with L2 regularisation (sklearn default). Produces calibrated probability estimates. Coefficients are directly interpretable as feature weights.
- **Gaussian Naïve Bayes** — generative classifier that assumes features are conditionally independent given the class label. Fast and simple.

Evaluation

Evaluate both models on the evaluation set using bootstrap resampling (1000 resamples, 95% CI). Report:

- **Accuracy** — fraction of correctly classified samples
- **F1 score** — harmonic mean of precision and recall (binary)
- **MCC** — Matthews Correlation Coefficient (robust to class imbalance)
- **ROC-AUC** — area under the receiver operating characteristic curve
- **PR-AUC** — area under the precision–recall curve

Deliverable (Bonus B): Required table:

Model	Accuracy (95% CI)	F1 (95% CI)	MCC (95% CI)	ROC-AUC	Features

Also include: confusion matrices for both models, ROC curves on the same axes, bootstrap boxplots for all four metrics, and a bar chart of the top 20 sex-discriminative CpGs by |point-biserial r|. Discuss why Logistic Regression and Gaussian NB perform differently on this data.

Grading Scheme

The assignment is graded on a 100-point scale. Bonus tasks are totally optional, graded separately. You receive two total grades: one for the required part and one for the Bonus part. If you accumulate a substantial amount of Bonus points during the course, your final class grade may increase by up to a full mark (e.g., a 7.2 may become 8). It also plays a role if you ask for a reference letter in the future.

Task / Subtask	Pts	Grading criteria
Task 1: Preprocessing	**15**	
1.1 Data splitting	5	Correct 80/20 split, stratified by age, seed=42. Evaluation set kept locked.
1.2 Preprocessing pipeline	5	Missing values handled with justified strategy, no leakage. Scaling applied in Pipeline. Categorical features handled appropriately. Feature matrices constructed and motivated.
1.3 Exploratory analysis	5	Dataset statistics table, age distribution plot, justification of preprocessing choices.
Task 2: Baseline models	**20**	
2.1 OLS across feature sets	8	Meaningful feature set combinations evaluated, bootstrap protocol correct, results table with 95% CIs, interpretation of what each feature set reveals.
2.2 Three models at defaults	12	All three models trained, evaluated, and compared to OLS floor. Boxplots included.
Task 3: Feature selection	**25**	
3.1 Stability selection	10	Correct bootstrap procedure, stability scores reported, feature frequency distribution shown.
3.2 mRMR (chosen K)	10	Correct implementation, K justified with validation-set reasoning, top-10 features reported, overlap visualisation included.
3.3 Method comparison	5	Validation RMSE comparison, method selected with justification.
Task 4: Tuning & evaluation	**40**	
4.1 RandomizedSearchCV	15	Correct CV setup (5-fold, dev set only), 40+ iterations, search spaces as specified.
4.2 Final evaluation	15	Bootstrap evaluation on eval set, all metrics with 95% CIs, tables in required format, boxplots and scatter plots.
4.3 Model selection	10	Best model justified with reasoning covering performance, complexity, and interpretability.
Report quality	—	Max 10 pages. Clear structure. Results presented in required table format. All figures labelled. GitHub link included.

Bonus A: Optuna	**20**	Head-to-head table, optimisation history figure, discussion of when TPE offers advantage.
Bonus B: Sex prediction	**30**	Correct mRMR re-selection, both classifiers evaluated, all required figures and tables present, comparison of the two models.
Total	**100**	(+50 bonus maximum)

Deliverables

Submit via eclass by the deadline. Your submission must include:

1. **PDF report** (maximum 10 pages, typed) summarising your methods and results in the form of a technical article. The report must include:
    - **Brief Introduction** of the background of the problem and the goals of the analysis
    - **Materials and methods:** Describe the dataset, the preprocessing, baseline models, feature selection, regression (and classification, if you do the bonus) models, and the evaluation procedures
    - **Results:** Present the results of your experiments. Include relevant tables and figures
    - **Discussion:** Interpret the results and discuss model performance, biological interpretation and limitations of the study
    - **Conclusions**

2. **GitHub repository** containing:
    - `src/functions.py` — all pipeline functions
    - `notebooks/data_exploration.ipynb` and `notebooks/model_analysis.ipynb`
    - `models/best_model.pkl` — saved best model

    The repository should be clean, reproducible and well organized.

Hints and Best Practices

- **Single Responsibility Principle:** Each function in functions.py should do one thing. Use the separation of concerns principle.
- **No leakage:** imputation, scaling, and feature selection statistics must be computed on the training split only and applied to val/eval sets.
- **Reproducibility:** use seed=42 everywhere. Your results should be exactly reproducible by running notebooks top-to-bottom.
- **Pipelines:** use sklearn Pipelines so that preprocessing is applied consistently during cross-validation.
- **Bootstrap CI:** resample with replacement from the evaluation set 1000 times to estimate metrics uncertainty. Report the 2.5th and 97.5th percentile as your 95% CI.
- **Figure quality:** label all axes, include titles and units, and save at 150 dpi minimum.
- **Start early:** the feature selection and tuning steps can take significant runtime. Always test your code on a small subset first.

All preprocessing steps (e.g., imputation, encoding, scaling, or feature selection) must be implemented within a single scikit-learn `Pipeline` or equivalent workflow. The pipeline should be **fit only on the training data**, ensuring that preprocessing parameters are learned exclusively from the training subset. This is required because your submitted model will be evaluated on a **separate hold-out CSV** by the reviewers, and the pipeline must correctly apply the same preprocessing to unseen data without refitting.

A Note on LLM Usage

Large language model tools are widely used in academia and industry, let's not pretend otherwise. You can of course use them for learning aid and assistance. However, you should keep in mind that there are levels to this. In the end, **you are responsible for the code you submit and the deliverables you write.**
Using LLMs to generate code you cannot understand will leave you unable to debug it and more importantly maintain it, which is the biggest problem. It will also make it more difficult for you to interpret results, answer questions about your implementations and you will also forget most of it soon after.
Being a Data Scientist is mostly about being a scientist; meaning being able to hypothesize, test, understand and know about the fundamentals, not just produce blocks of code - and this is what is going to separate you from the rest with access to LLM premium models.

Remember also to never blindly trust LLMs as their probabilistic nature makes them prone to errors and hallucinations that look true. Always verify.

Finally, for this course it is required to disclose LLM usage. Please include an "AI use" statement at the end of your report in case you use them for code or report writing.

Deadline: Monday March 30th, 11:59pm

Submission via eclass. Start early — the devil is in the details!